Step 1: Set Up Environment and Load Libraries

```
# Import necessary libraries
import cv2
import numpy as np
import tensorflow as tf
import matplotlib.pyplot as plt
from tensorflow.keras.applications import ResNet50
from tensorflow.keras.models import Sequential
from tensorflow.keras.layers import Dense, Flatten
from tensorflow.keras.optimizers import Adam
from sklearn.model_selection import train_test_split
from google.colab import drive
from google.colab.patches import cv2_imshow
import os

# Mount Google Drive for dataset
drive.mount('/content/drive')
```

```
Mounted at /content/drive
```

Step 2: Download and Prepare the PASCAL VOC Dataset

```
# Define paths to images and annotations (adjust these if using Drive)
# Download VOC2012 if not done
!wget http://host.robots.ox.ac.uk/pascal/VOC/voc2012/VOCtrainval_11-May-2012.tar
!tar -xf VOCtrainval_11-May-2012.tar

# Define the image and annotation paths
IMAGE_PATH = '/content/VOCdevkit/VOC2012/JPEGImages/'
ANNOTATION_PATH = '/content/VOCdevkit/VOC2012/Annotations/'
```

```
--2024-11-04 04:06:13--  http://host.robots.ox.ac.uk/pascal/VOC/voc2012/VOCtrainval_11-May-2012.tar
Resolving host.robots.ox.ac.uk (host.robots.ox.ac.uk)... 129.67.94.152
Connecting to host.robots.ox.ac.uk (host.robots.ox.ac.uk)|129.67.94.152|:80... connected.
HTTP request sent, awaiting response... 200 OK
Length: 1999639040 (1.9G) [application/x-tar]
Saving to: ‘VOCtrainval_11-May-2012.tar’

VOCtrainval_11-May- 100%[===================>]   1.86G  29.3MB/s    in 70s

2024-11-04 04:07:23 (27.3 MB/s) - ‘VOCtrainval_11-May-2012.tar’ saved [1999639040/1999639040]
```

Step 3: Parse Annotations and Load Images with Labels

```
import xml.etree.ElementTree as ET

# Parse PASCAL VOC annotations
def parse_voc_annotation(annotation_path):
    tree = ET.parse(annotation_path)
    root = tree.getroot()
    boxes = []
    labels = []

    for obj in root.findall('object'):
        label = obj.find('name').text
        bbox = obj.find('bndbox')
        xmin = int(bbox.find('xmin').text)
        ymin = int(bbox.find('ymin').text)
        xmax = int(bbox.find('xmax').text)
        ymax = int(bbox.find('ymax').text)
        boxes.append((xmin, ymin, xmax, ymax))
        labels.append(label)

    return boxes, labels

# Example: Load images and annotations
image_files = os.listdir(IMAGE_PATH)[:100]  # Limiting to 100 images for speed
images = []
annotations = []

for image_file in image_files:
    image = cv2.imread(os.path.join(IMAGE_PATH, image_file))
    images.append(image)
```

```
    annot_path = os.path.join(ANNOTATION_PATH, image_file.replace('.jpg', '.xml'))
    boxes, labels = parse_voc_annotation(annot_path)
    annotations.append((boxes, labels))
```

Step 4: Generate Region Proposals Using Selective Search

```
def selective_search(image):
    ss = cv2.ximgproc.segmentation.createSelectiveSearchSegmentation()
    ss.setBaseImage(image)
    ss.switchToSelectiveSearchQuality()  # Quality mode for accuracy

    # Process and get bounding box proposals
    rects = ss.process()
    regions = []
    for (x, y, w, h) in rects:
        if w > 50 and h > 50:  # Filter small regions for PASCAL VOC
            regions.append((x, y, x + w, y + h))
    return regions
```

Step 5: Extract and Resize Regions for Feature Extraction

```
def generate_proposals(images, annotations):
    proposals = []
    labels = []

    for i, img in enumerate(images):
        regions = selective_search(img)
        boxes, img_labels = annotations[i]

        for (x1, y1, x2, y2) in regions[:100]:  # Limiting to 100 regions per image
            region = img[y1:y2, x1:x2]
            if region.shape[0] < 64 or region.shape[1] < 64:
                continue  # Skip small regions

            region_resized = cv2.resize(region, (64, 64))
            proposals.append(region_resized)
            labels.append(img_labels[0])  # Simplifying by taking the first label

    return np.array(proposals), np.array(labels)

# Generate region proposals
train_proposals, train_labels = generate_proposals(images, annotations)
```

Step 6: Feature Extraction Using ResNet50

```
# Load ResNet50 with ImageNet weights
resnet = ResNet50(weights='imagenet', include_top=False, input_shape=(64, 64, 3))
resnet.trainable = False  # Freeze ResNet50 layers

# Define feature extractor model
feature_extractor = Sequential([
    resnet,
    Flatten(),
])

# Extract features
train_features = feature_extractor.predict(train_proposals)
```

```
Downloading data from https://storage.googleapis.com/tensorflow/keras-applications/resnet/resnet50_weights_tf_dim_ordering_tf_kernel
94765736/94765736 ━━━━━━━━━━━━━━━━━━━━ 1s 0us/step
232/232 ━━━━━━━━━━━━━━━━━━━━ 10s 26ms/step
```

Step 7: Train the Classifier on Extracted Features

```
from sklearn.preprocessing import LabelEncoder
from tensorflow.keras.utils import to_categorical

# Encode class labels
label_encoder = LabelEncoder()
encoded_labels = label_encoder.fit_transform(train_labels)
encoded_labels = to_categorical(encoded_labels)
```

```
# Define classifier model
classifier = Sequential([
    Dense(512, activation='relu', input_shape=(train_features.shape[1],)),
    Dense(256, activation='relu'),
    Dense(len(label_encoder.classes_), activation='softmax')
])

# Compile classifier
classifier.compile(optimizer=Adam(learning_rate=0.001), loss='categorical_crossentropy', metrics=['accuracy'])

# Train classifier on features
classifier.fit(train_features, encoded_labels, epochs=20, batch_size=32, validation_split=0.1)
```

```
/usr/local/lib/python3.10/dist-packages/keras/src/layers/core/dense.py:87: UserWarning: Do not pass an `input_shape`/`input_dim` arg
  super().__init__(activity_regularizer=activity_regularizer, **kwargs)
Epoch 1/20
209/209 ━━━━━━━━━━━━━━━━━━━━ 7s 18ms/step - accuracy: 0.6130 - loss: 2.1683 - val_accuracy: 0.2332 - val_loss: 8.7342
Epoch 2/20
209/209 ━━━━━━━━━━━━━━━━━━━━ 1s 3ms/step - accuracy: 0.9530 - loss: 0.1716 - val_accuracy: 0.2507 - val_loss: 10.7987
Epoch 3/20
209/209 ━━━━━━━━━━━━━━━━━━━━ 1s 3ms/step - accuracy: 0.9784 - loss: 0.0670 - val_accuracy: 0.2817 - val_loss: 12.1362
Epoch 4/20
209/209 ━━━━━━━━━━━━━━━━━━━━ 1s 3ms/step - accuracy: 0.9902 - loss: 0.0440 - val_accuracy: 0.2615 - val_loss: 13.1555
Epoch 5/20
209/209 ━━━━━━━━━━━━━━━━━━━━ 1s 3ms/step - accuracy: 0.9612 - loss: 0.1809 - val_accuracy: 0.2102 - val_loss: 11.7928
Epoch 6/20
209/209 ━━━━━━━━━━━━━━━━━━━━ 1s 3ms/step - accuracy: 0.9830 - loss: 0.0639 - val_accuracy: 0.2439 - val_loss: 15.8235
Epoch 7/20
209/209 ━━━━━━━━━━━━━━━━━━━━ 1s 3ms/step - accuracy: 0.9649 - loss: 0.1847 - val_accuracy: 0.2830 - val_loss: 16.3487
Epoch 8/20
209/209 ━━━━━━━━━━━━━━━━━━━━ 1s 3ms/step - accuracy: 0.9852 - loss: 0.0609 - val_accuracy: 0.2615 - val_loss: 14.9350
Epoch 9/20
209/209 ━━━━━━━━━━━━━━━━━━━━ 1s 3ms/step - accuracy: 0.9915 - loss: 0.0292 - val_accuracy: 0.2857 - val_loss: 14.8433
Epoch 10/20
209/209 ━━━━━━━━━━━━━━━━━━━━ 1s 3ms/step - accuracy: 0.9885 - loss: 0.0388 - val_accuracy: 0.2628 - val_loss: 15.4064
Epoch 11/20
209/209 ━━━━━━━━━━━━━━━━━━━━ 1s 3ms/step - accuracy: 0.9993 - loss: 0.0025 - val_accuracy: 0.2480 - val_loss: 15.5668
Epoch 12/20
209/209 ━━━━━━━━━━━━━━━━━━━━ 1s 3ms/step - accuracy: 0.9984 - loss: 0.0068 - val_accuracy: 0.2588 - val_loss: 16.0751
Epoch 13/20
209/209 ━━━━━━━━━━━━━━━━━━━━ 1s 3ms/step - accuracy: 1.0000 - loss: 1.0912e-04 - val_accuracy: 0.2574 - val_loss: 15.9071
Epoch 14/20
209/209 ━━━━━━━━━━━━━━━━━━━━ 1s 4ms/step - accuracy: 1.0000 - loss: 5.6246e-05 - val_accuracy: 0.2588 - val_loss: 16.0728
Epoch 15/20
209/209 ━━━━━━━━━━━━━━━━━━━━ 2s 6ms/step - accuracy: 1.0000 - loss: 4.3562e-05 - val_accuracy: 0.2642 - val_loss: 16.1725
Epoch 16/20
209/209 ━━━━━━━━━━━━━━━━━━━━ 2s 7ms/step - accuracy: 1.0000 - loss: 3.3829e-05 - val_accuracy: 0.2642 - val_loss: 16.2453
Epoch 17/20
209/209 ━━━━━━━━━━━━━━━━━━━━ 1s 4ms/step - accuracy: 1.0000 - loss: 3.5444e-05 - val_accuracy: 0.2642 - val_loss: 16.3103
Epoch 18/20
209/209 ━━━━━━━━━━━━━━━━━━━━ 1s 4ms/step - accuracy: 1.0000 - loss: 2.5885e-05 - val_accuracy: 0.2642 - val_loss: 16.3714
Epoch 19/20
209/209 ━━━━━━━━━━━━━━━━━━━━ 1s 3ms/step - accuracy: 1.0000 - loss: 2.5320e-05 - val_accuracy: 0.2642 - val_loss: 16.4297
Epoch 20/20
209/209 ━━━━━━━━━━━━━━━━━━━━ 1s 3ms/step - accuracy: 1.0000 - loss: 2.0304e-05 - val_accuracy: 0.2642 - val_loss: 16.4901
<keras.src.callbacks.history.History at 0x7b802f6f41c0>
```

Step 8: Evaluate Model and Display Predictions

```
from sklearn.metrics import accuracy_score

def display_predictions_with_accuracy(images, features, true_labels, classifier, label_encoder):
    # Make predictions on the test features
    predictions = classifier.predict(features)
    predicted_classes = np.argmax(predictions, axis=1)
    decoded_predictions = label_encoder.inverse_transform(predicted_classes)

    # Calculate accuracy
    true_encoded_labels = label_encoder.transform(true_labels)
    accuracy = accuracy_score(true_encoded_labels, predicted_classes)
    print(f"Model Test Accuracy: {accuracy:.4f}")

    # Display sample predictions
    plt.figure(figsize=(12, 12))
    for i in range(9):  # Display 9 samples
        plt.subplot(3, 3, i + 1)
        plt.imshow(images[i])
        plt.title(f"Predicted: {decoded_predictions[i]}")
        plt.axis("off")
    plt.show()

# Generate test proposals and labels
```

```
test_proposals, test_labels = generate_proposals(images[:20], annotations[:20])
test_features = feature_extractor.predict(test_proposals)

# Display predictions and accuracy
display_predictions_with_accuracy(test_proposals, test_features, test_labels, classifier, label_encoder)
```

```
47/47 ━━━━━━━━━━━━━━━━━━━━ 2s 48ms/step
47/47 ━━━━━━━━━━━━━━━━━━━━ 0s 6ms/step
Model Test Accuracy: 0.9417
```